



Asset Performance Management

PI Data Cache Service

User Guide

Release 540.1

August 2023

DISCLAIMER

This document contains Honeywell proprietary information. Information contained herein is to be used solely for the purpose submitted, and no part of this document or its contents shall be reproduced, published, or disclosed to a third party without the express permission of Honeywell International Sàrl.

While this information is presented in good faith and believed to be accurate, Honeywell disclaims the implied warranties of merchantability and fitness for a purpose and makes no express warranties except as may be stated in its written agreement with and for its customer.

In no event is Honeywell liable to anyone for any direct, special, or consequential damages. The information and specifications in this document are subject to change without notice.

Copyright 2023 - Honeywell International Sàrl

CONTENTS

- Chapter 1 - Introduction 4
- Chapter 2 - Topology 5
- Chapter 3 - Pre-requisites 6
- Chapter 4 - Installation 7
- Chapter 5 - Uninstallation 8
- Chapter 6 - Application Settings 9
- Chapter 7 - PI Data Cache API Details 10
 - 7.1 REST API Endpoints10
 - 7.2 PIDataCache client 15

INTRODUCTION

The PI Data Cache Service cache real-time data. It is available to APM Python calculations and helps in writing data to PI historian.

Installation: Install the software on Application Server.

PRE-REQUISITIES

- Microsoft .Net framework v4.8
- Download and install the PI AF Client (it can be downloaded from PI OSI software center).

INSTALLATION

Follow the below steps for PI Data Cache Service installation:

1. In the Application Server with Asset Performance Management media, navigate to the following folder.
<ISO>\Utilities\Honeywell.UAS.PIDataCacheService
2. Create PI Data Cache Service folder in one of the system drives.
3. Copy the binary files from media to the created folder.
4. Open command prompt in administrator mode.
5. Run the below command:
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\installutil.exe "[path to
Honeywell.UAS.PIDataCacheService.exe]"
6. Enter domain username and password in the dialog box.
Click **Continue**.
7. The UAS PI Data Cache Service is created in **Services.msc**.

UNINSTALLATION

Perform the following steps to uninstall the PI Data Cache Service:

1. Choose **Start > Run**, type **Services.msc** and click **OK**.
The **Services** window appears.
2. In the Services panel, find **Honeywell PI Data Cache Service**, right-click and select **Stop**.
3. In the Application Server, go to the installed location of PI Data Cache Service.
4. Open command prompt in administrator mode.
5. Run the below command:
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\installutil.exe /u "[path to Honeywell.UAS.PIDataCacheService.exe]"
6. The UAS PI Data Cache Service is removed from **Services.msc**.

APPLICATION SETTINGS

Follow below steps to change the Application Settings parameter values:

1. Open the created Calculation Logger folder where binary files are copied.
2. Open **.exe.config**.
3. Locate **<appSettings>** section in **exe.config** file.
4. Update the **PI Server Name** and **Server name**.

```
<CacheServiceConfig>
<PIServers>
<PIServer Name="" Server=""/>
</PIServers>
</CacheServiceConfig>
```

Parameter	Description
PI Server Name	It is same as Data Source name in APM Data Source configuration.
Server	It is actual PI Server IP or fully qualified PI server name can also be used here.

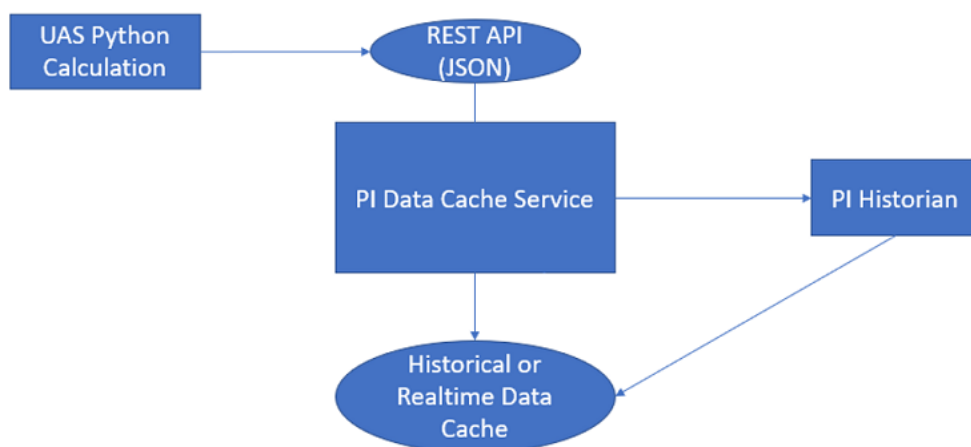
Example:

```
<CacheServiceConfig>
<PIServers>
<PIServer Name="AUSPICOLL" Server="10.78.81.230"/>
<PIServer Name="CANPICOLL" Server="10.78.81.231"/>
<PIServer Name="DSAMPICOLL" Server="10.78.81.232"/>
<PIServer Name="EUACCPICOLL" Server="10.78.81.233"/>
<PIServer Name="DSEUPICOLL" Server="10.78.81.234"/>
<PIServer Name="NIGPICOLL" Server="10.78.81.235"/>
<PIServer Name="SAPICOLL" Server="10.78.81.236"/>
<PIServer Name="UPPICOLL" Server="10.78.81.237"/>
</PIServers>
</CacheServiceConfig>
```

PI DATA CACHE API DETAILS

Python data-driven calculation utilizing effective historical or real time data caching through PI Data Cache Service.

The flow diagram of the above explanation is given below:



7.1 REST API Endpoints

PI Data Cache Service is a Windows service which exposes the following REST API endpoints:

1. **GetData:** For requesting a historical PI tag data value or list of values for the selected time period and place it in cache.

`https://<hostname>/UASPIDataCacheService/GetData?tag=×tamp=×tamp2=&cachePolicy=&aggregate=&removeBadData=&context=&doInterp=`

Example:

```
https://hostname/UASPIDataCacheService/GetData?tag=DSAMPICOLL\SINUSOID&timestamp=2020-04-29
02:00:00&cachePolicy=2h&aggregate=plot250&removeBadData=true&context=mycalcl&doInterp=false
```

```
Result: {"GetDataResult":{"Error":"","Value":{"Quality":192,"Timestamp":"2020-04-29
02:10:00","Value":0.1722874},"Values":null}}|
```

```
https://hostname/UASPIDataCacheService/GetData?tag=DSAMPICOLL\SINUSOID&timestamp=2020-04-29
02:48:00&timestamp2=2020-04-29
02:50:00&cachePolicy=2h&aggregate=plot250&removeBadData=true&context=mycalcl&doInterp=false
```

```
Result: {"GetDataResult":{"Error":"","Value":null,"Values":[{"Quality":192,
"Timestamp":"2020-04-29 02:48:00","Value":4.237154}, {"Quality":192,"Timestamp":"2020-04-29
02:48:01","Value":4.325482}, {"Quality":192,"Timestamp":"2020-04-29
02:48:31","Value":4.41468}, {"Quality":192,"Timestamp":"2020-04-29
02:49:01","Value":4.504745}, {"Quality":192,"Timestamp":"2020-04-29
02:49:31","Value":4.595677}]}By forwardly requesting more data than requested from PI
historian and caching them, later requests can get data from cache directly if the
timestamp falls in the time range of the cached data.
```

Descriptions of QueryString parameters of **GetData** endpoint are given below:

Parameters	Description
tag	The tag format is PIServerKey\tagname. The PIServerKey is a configurable key for a PI server in the service's configuration file.
timestamp	The UTC time of the requested historical PI tag data value.
timestamp2	It is an optional UTC time to request the historical data of a particular time range. The data between timestamp and timestamp2 is retrieved.
cachePolicy	Forwardly request more tag data from PI historian than the specified UTC timestamp and place it in cache. New request can retrieve tag data from cache directly, if the timestamp falls in the time range of data cached during forwardly request. The cachePolicy format is nM, nH or nD. Where, n = Number. M = Minutes. H = Hours. D = Days. Example: 40M means requesting tag data from timestamp to timestamp+40 minutes from historian and place them in the cache, if the data at timestamp is not already available in the cache. NOTE CachePolicy need to be big enough for historical data caching. If it is small use realtime caching.
aggregate	The aggregate format is plotn. Where, n = plot value. Example: aggregate=plot250 NOTE The minimum plot value is 50.
removeBadData	The data of bad quality is removed. By default the value is true .
context	Context + PIServerKey + tag is used as key of a cached item.

Parameters	Description
	NOTE The context need to be specific, so one cached item is not used by different calculations. Context can be generated from calculation and AssetGUID. Example: APSSIB-A: 1786dfc0 (for APS_SIBehavior_Calc_AnnulusAPressure), APSSIB-B-1786dfc0 (APS_SIBehavior_Calc_AnnulusBPressure), BleedDownROC-A-1786dfc0 (APS_BleedDown_AnnulusAPressure_Calc), WellboreDP-1786dfc0 (OGS_WellboreDP_Calc). In the above example, 1786dfc0 is the first 8 characters of AssetGUID.
doInterp	If the value is true, time average is used to retrieve the tag values for the selected timestamp from the cache. If the value is false, the previous value of timestamp is returned from the cache. NOTE Use false if the tags have discrete values. An example for discrete value is digital state tags.

2. **CacheAndGetData**: For caching a real time data and requesting a historical value or a list of values for the selected time period.

<https://<hostname>/UASPIDataCacheService/CacheAndGetData?tag=×tamp=×tamp2=&cachevalue=&cachetimestamp=&removeBadData=&context=&doInterp=>

Example:

```
https://hostname/UASPIDataCacheService/CacheAndGetData?tag=DSAMPICOLL\SINUSOID&timestamp=2020-04-29 02:23:00&cachevalue=7.2&cachetimestamp=2020-04-29 02:25:00&removeBadData=true&context=mycalc1&doInterp=false
Result: {"CacheAndGetDataResult":{"Error":"","Value":{"Quality":192,"Timestamp":"2020-04-29 02:23:00","Value":6.8},"Values":null}}
```

Descriptions of QueryString parameters of **CacheAndGetData** endpoint are given below:

Parameters	Description
tag	The Tag format is PIServerKey\tagname. The PIServerKey is a configurable key for a PI server in the service's configuration file.
timestamp	The UTC time of the requested historical PI tag data value.
timestamp2	It is an optional UTC time to request the historical data of a particular time range. The data between timestamp and timestamp2 is retrieved.
cachevalue	It is the realtime value to be cached. The cachevalue can be empty if there is no need for realtime value to be cached or if the value is not numerical.

Parameters	Description
cachetimestamp	The timestamp of value to be cached.
removeBadData	The data of bad quality is removed. By default the value is true .
context	Context + PIServerKey + tag is used as key of a cached item. NOTE The context need to be specific, so one cached item is not used by different calculations.
doInterp	If the value is true , time average is used to retrieve the tag values for the selected timestamp from the cache. If the value is false , the previous value of timestamp is returned from the cache. NOTE Use false if the tags have discrete values. An example for discrete value is digital state tags.

3. **GetValues**: For requesting historical PI tag data value without caching.

<https://<hostname>/UASPIDataCacheService/GetValues?tag=&start=&end=&aggregate=&removeBadData=>

Example:

```
https://hostname/UASPIDataCacheService/GetValues?tag=DSAMPICOLL\SINUSOID&start=2020-04-29 02:40:00&end=2020-04-29 02:50:00&aggregate=interpolative5&removeBadData=true&context=mycalc1

Result: {"GetValuesResult":{"Error":"","Values":[{"Quality":192,"Timestamp":"2020-04-29 02:40:00","Value":3.015213}, {"Quality":192,"Timestamp":"2020-04-29 02:42:00","Value":3.320815}, {"Quality":192,"Timestamp":"2020-04-29 02:44:00","Value":3.638001}, {"Quality":192,"Timestamp":"2020-04-29 02:46:00","Value":3.974483}, {"Quality":192,"Timestamp":"2020-04-29 02:48:00","Value":4.322538}, {"Quality":192,"Timestamp":"2020-04-29 02:50:00","Value":4.684413}]}}
```

Descriptions of QueryString parameters of **GetValues** endpoint are given below:

Parameters	Description
tag	The Tag format is PIServerKey\tagname . The PIServerKey is a configurable key for a PI server in the service's configuration file.
start, end	The UTC time range of the requested tag values.
aggregate	The different formats used are: 1. RAWn format. Example: raw500 retrieves maximum of 500 raw data. 2. In PLOTn format.

Parameters	Description
	Example: <pre>plot 250</pre> 3. In INTERPOLATIVEn. Example: <pre>interpolative5 to retrieve 5 values.</pre> 4. In INTERPOLATIVEnS/M/H/D format. Example: <pre>interpolative5m to interpolate values every 5 minutes.</pre> 5. In AVERAGEnS/M/H/D format. Example: <pre>average5m.</pre>
removeBadData	The data of bad quality is removed. By default the value is true .
context	It is an optional QueryString parameter that help you identify errors in log file.

4. **GetValuesMultiple**: For requesting historical data of multiple tags without caching.

<https://<hostname>/UASPIDataCacheService/GetValues?tag=&start=&end=&aggregate=&removeBadData=>

Example:

```
https://hostname/UASPIDataCacheService/GetValuesMultiple?tag=DSAMPICOLL\SINUSOID&start=2020-04-29 02:40:00&end=2020-04-29 02:50:00&aggregate=interpolative5&removeBadData=true&context=mycalc1
```

Result:

```
{ "GetValuesMultipleResult": { "Error": "", "ValuesList": [ { "Quality": 192, "Timestamp": "2020-04-29 02:40:00", "Value": 3.015213 }, { "Quality": 192, "Timestamp": "2020-04-29 02:42:00", "Value": 3.320815 }, { "Quality": 192, "Timestamp": "2020-04-29 02:44:00", "Value": 3.638001 }, { "Quality": 192, "Timestamp": "2020-04-29 02:46:00", "Value": 3.974483 }, { "Quality": 192, "Timestamp": "2020-04-29 02:48:00", "Value": 4.322538 }, { "Quality": 192, "Timestamp": "2020-04-29 02:50:00", "Value": 4.684413 } ] } }
```

Descriptions of QueryString parameters of **GetValuesMultiple** endpoint are given below:

Parameters	Description
tag	The values are returned in ValuesList element. The Tag format is PIServerKey\tag1name, tag2name, tag3name, etc.

NOTE

Other parameters used in **GetValuesMultiple** are same as **GetValues**.

5. **WriteValues**: For writing PI tag data (POST)

<https://<hostname>/UASPIDataCacheService/WriteValues?context=>

NOTE

Content for tag values to post should be in json format.

Example:

```
Post: {"tag":"server1\\tag1","values":[{"value":1.2, "timestamp":"2020-04-20
01:00:00"}, {"value":1.3, "timestamp":"2020-04-20 01:05:00"}]} (notice \ escaping for
tag)
```

```
The above dumped as string (notice \ escaping for tag):
'{"tag":"server1\\\\tag1","values":[{"value":1.2, "timestamp":"2020-04-20
01:00:00"}, {"value":1.3, "timestamp":"2020-04-20 01:05:00"}]}'
```

```
Result: {"WriteValuesResult":{"Error":""}}
```

Descriptions of QueryString parameters of **WriteValues** endpoint are given below:

Parameters	Description
Context	It is an optional QueryString parameter that help you identify errors in log file.
POST body	Allows you to save multiple values for a tag in UTC Timestamp. The Tag is of the format PIServerKey\tagname Example: <pre>{ "tag": "tag1", "values": [{ "value": 1.1, "timestamp": "2020-04-20 11:00:00" }, { "value": 1.2, "timestamp": "2020-04-20 11:05:00" }, { "value": 1.3, "timestamp": "2020-04-20 11:10:00" }] }</pre>

7.2 PIDataCache client

The `as_pidatacache_client.py` is deployed at APM Calculation server's PRE directory for APM Python calculation to invoke the REST API by read (invoke `GetData`) method, `cacheAndRead`(invoke `CacheAndGetData`) method, and `readValues` (invoke `GetValues`) method with a dictionary of parameters mentioned above.

NoEncode version of these methods (`readNoEncode`, `cacheAndReadNoEncode`, `readValuesNoEncode`) are provided when parameter encoding is not desired. It also invokes `WriteValues` by `writeValues` method via POST.

NOTICES

Terms and Conditions

Honeywell International Inc. authorizes you to copy documents published by Honeywell International Inc. on the World Wide Web for personal or non-commercial use only, provided any copy of these documents that you make shall retain all copyright and other proprietary notices contained herein. Except as expressly provided, nothing contained in this paragraph shall be construed as conferring any license or right under any Honeywell International Inc. copyright. No materials available on the Honeywell International Inc. Website may be stored, transmitted by any means (including but not limited to electronic, mechanical, scanning, photocopying or recording) without prior written permission of Honeywell International Inc. For more details, refer to <https://www.honeywell.com/us/en/terms-and-conditions>.

Other trademarks

Microsoft and SQL Server are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.

Trademarks that appear in this document are used only to the benefit of the trademark owner, with no intention of trademark infringement.

Third-party licenses

This product may contain or be derived from materials, including software, of third parties. The third party materials may be subject to licenses, notices, restrictions and obligations imposed by the licensor. The licenses, notices, restrictions and obligations, if any, may be found in the materials accompanying the product, in the documents or files accompanying such third party materials, in a file named third_party_licenses on the media containing the product.

Documentation feedback

You can find the most up-to-date documents on the Honeywell Process Solutions support website at:

<http://www.honeywellprocess.com/support>

If you have comments about Honeywell Connected Industrial documentation, send your feedback to:

hpsdocs@honeywell.com

Use this email address to provide feedback, or to report errors and omissions in the documentation. For immediate help with a technical problem, contact your local Honeywell Process Solutions Customer Contact Center (CCC) or Honeywell Technical Assistance Center (TAC).

How to report a security vulnerability

For the purpose of submission, a security vulnerability is defined as a software defect or weakness that can be exploited to reduce the operational or security capabilities of the software.

Honeywell investigates all reports of security vulnerabilities affecting Honeywell products and services.

To report a potential security vulnerability against any Honeywell product, please follow the instructions at:

<https://honeywell.com/pages/vulnerabilityreporting.aspx>

Support

For support, contact your local Honeywell Process Solutions Customer Contact Center (CCC). To find your local CCC visit the website, <https://www.honeywellprocess.com/en-US/contact-us/customer-support-contacts/Pages/default.aspx>.

Training classes

Honeywell holds technical training classes that are taught by process control systems experts. For more information about these classes, contact your Honeywell representative, or see <http://www.automationcollege.com>.